

Assignment V: Branch Prediction and Speculative Execution in a Mini-RISC Processor

Department of Computer Science and Engineering

1. Background and Motivation

So far, you have built a pipelined Mini-RISC processor with:

- Instruction pipelining
- Hazard detection and forwarding
- Cache for memory performance

However, **branch instructions** still reduce performance.

A branch instruction decides the **next value of PC**. But this decision is known only in the **EX stage**.

This creates a **control hazard**.

2. What Is a Control Hazard?

Consider the code:

```
CMP R0
JZ 20
ADD R1
```

The processor must decide:

- Should PC go to the next instruction?
- Or should PC jump to address 20?

But the condition (Z flag) is known only after **CMP** executes.

Without prediction, the processor:

- Waits
- Or fetches incorrectly
- Then flushes the pipeline

Flushing wastes cycles.

3. Basic Pipeline Timeline (No Prediction)

Cycle 1: IF(CMP)
Cycle 2: EX(CMP) | IF(JZ)
Cycle 3: EX(JZ) | IF(ADD) <-- wrong fetch if jump taken
Cycle 4: FLUSH
Cycle 5: IF(target)

Each taken branch costs multiple cycles.

4. Key Idea: Branch Prediction

Instead of waiting, the processor:

- **Predicts** whether the branch will be taken
- Fetches instructions **speculatively**
- Corrects itself only if the prediction was wrong

Important: Prediction affects only **performance**, not correctness.

5. Static Predictor: Predict-Not-Taken

This is the simplest predictor:

- Always assume branch is **not taken**
- Fetch the next sequential instruction

This works well because:

- Many branches are not taken
- Loop-exit branches are taken only once

6. Dynamic Predictor: 1-Bit Predictor

A 1-bit predictor remembers the last outcome of a branch.

- 0 → predict not taken
- 1 → predict taken

Each branch instruction has one bit of history.

Update rule:

- After branch executes, update the bit with actual outcome

7. Worked Example: Loop Branch

Consider:

```
LOOP:  
ADD R1  
CMP R1  
JZ LOOP
```

Assume the loop runs 10 times.

Without Prediction

- Branch taken 9 times
- Pipeline flushed 9 times

With Predict-Not-Taken

- First 9 iterations mispredicted
- Last iteration correct

With 1-Bit Predictor

- First iteration mispredicted
- Predictor learns “taken”
- Next 8 iterations correct
- Final iteration mispredicted

Huge reduction in flushes.

8. Speculative Execution in Your Processor

In your Mini-RISC CPU:

- IF stage uses the predictor to choose next PC
- EX stage checks the real outcome
- If prediction is wrong:
 - Flush pipeline
 - Correct PC

9. What You Must Implement

1. Branch Predictor Table
2. Prediction in IF stage
3. Branch resolution in EX stage
4. Pipeline flush on misprediction
5. Performance counters

10. Performance Counters

Your simulator must print:

```
Total instructions
Total branches
Correct predictions
Mispredictions
Misprediction penalty cycles
Total cycles
CPI
```

11. Complete C Skeleton: Branch Prediction and Speculation

```
1  /* =====
2  Mini-RISC CPU Simulator
3  Assignment V: Branch Prediction + Speculative Execution
4  -----
5  Supports:
6  (1) No Prediction
7  (2) Predict-Not-Taken
8  (3) 1-bit Branch Predictor
9  =====
10 */
11
12 #include <stdio.h>
13 #include <stdlib.h>
14 #include <string.h>
15
16 #define MEM_SIZE 256
17 #define BP_SIZE 16 // Branch Predictor Table size
18
19 /* ===== CONFIGURATION ===== */
20 enum {
21     NO_PREDICTION = 0,
22     PREDICT_NT = 1,
```

```

22     ONE_BIT_BP      = 2
23 };
24
25 int prediction_mode = ONE_BIT_BP;
26
27 /* ===== CPU STATE ===== */
28 typedef struct {
29     unsigned char R[4];
30     unsigned char ACC;
31     unsigned char PC;
32     unsigned char IR;
33     unsigned char Z, C;
34 } CPU;
35
36 CPU cpu;
37
38 /* ===== MEMORY ===== */
39 unsigned char IMEM[MEM_SIZE];
40
41 /* ===== PIPELINE REGISTER =====
42    */
43 typedef struct {
44     unsigned char IR;
45     unsigned char PC;
46     int valid;
47     int predicted_taken;
48 } IFEX;
49
50 IFEX ifex;
51
52 /* ===== BRANCH PREDICTOR =====
53    */
54 typedef struct {
55     int valid;
56     int prediction;    // 0 = Not Taken, 1 = Taken
57 } BPEnt;
58
59 BPEnt bp_table[BP_SIZE];
60
61 /* ===== PERFORMANCE COUNTERS
62    ===== */
63 int cycles = 0;
64 int instructions = 0;
65
66 int total_branches = 0;
67 int correct_predictions = 0;
68 int mispredictions = 0;
69 int branch_penalty_cycles = 0;
70
71 /* ===== UTILITY ===== */
72 void reset_cpu() {

```

```

70     memset(&cpu, 0, sizeof(CPU));
71     memset(&ifex, 0, sizeof(IFEX));
72     memset(bp_table, 0, sizeof(bp_table));
73 }
74
75 void load_program(const char *fname) {
76     FILE *fp = fopen(fname, "r");
77     if (!fp) { printf("Cannot open program file\n"); exit(1); }
78     int addr = 0;
79     unsigned int x;
80     while (fscanf(fp, "%x", &x) != EOF)
81         IMEM[addr++] = (unsigned char)x;
82     fclose(fp);
83 }
84
85 /* ===== INSTRUCTION HELPERS ===== */
86 int is_branch(unsigned char ir) {
87     unsigned char op = (ir >> 4) & 0xF;
88     return (op == 0xA || op == 0xB); // JMP or JZ
89 }
90
91 int branch_taken(unsigned char ir) {
92     unsigned char op = (ir >> 4) & 0xF;
93     if (op == 0xA) return 1; // JMP always taken
94     if (op == 0xB) return cpu.Z; // JZ depends on Z flag
95     return 0;
96 }
97
98 /* ===== BRANCH PREDICTOR ===== */
99 int bp_index(unsigned char pc) {
100     return pc % BP_SIZE;
101 }
102
103 int predict_branch(unsigned char pc) {
104     if (prediction_mode == NO_PREDICTION)
105         return 0;
106
107     if (prediction_mode == PREDICT_NT)
108         return 0;
109
110     int idx = bp_index(pc);
111     if (bp_table[idx].valid)
112         return bp_table[idx].prediction;
113
114     return 0; // default NT
115 }
116
117 void update_predictor(unsigned char pc, int taken) {
118     if (prediction_mode != ONE_BIT_BP)

```

```

119         return;
120
121         int idx = bp_index(pc);
122         bp_table[idx].valid = 1;
123         bp_table[idx].prediction = taken;
124     }
125
126     /* ===== PIPELINE STAGES ===== */
127     int flush_pipeline = 0;
128
129     void fetch_stage() {
130         ifex.IR = IMEM[cpu.PC];
131         ifex.PC = cpu.PC;
132         ifex.valid = 1;
133
134         if (is_branch(ifex.IR)) {
135             int pred = predict_branch(cpu.PC);
136             ifex.predicted_taken = pred;
137             cpu.PC = pred ? (ifex.IR & 0xF) : cpu.PC + 1;
138         } else {
139             ifex.predicted_taken = 0;
140             cpu.PC++;
141         }
142     }
143
144     void execute_stage() {
145         if (!ifex.valid) return;
146
147         unsigned char ir = ifex.IR;
148         unsigned char op = (ir >> 4) & 0xF;
149         unsigned char operand = ir & 0xF;
150
151         instructions++;
152
153         switch (op) {
154             case 0x3: cpu.ACC = operand; break;           // MOVI
155             case 0x4: cpu.ACC += cpu.R[operand]; break; // ADD
156             case 0x9: cpu.Z = (cpu.ACC == cpu.R[operand]); break; //
                        CMP
157
158             case 0xA: // JMP
159             case 0xB: // JZ
160             {
161                 total_branches++;
162                 int actual = branch_taken(ir);
163                 int predicted = ifex.predicted_taken;
164
165                 if (actual == predicted) {
166                     correct_predictions++;
167                 } else {
168                     mispredictions++;

```

```

169         branch_penalty_cycles += 2;
170         flush_pipeline = 1;
171         cpu.PC = actual ? operand : (ifex.PC + 1);
172     }
173     update_predictor(ifex.PC, actual);
174     break;
175 }
176
177 case 0xF:
178     printf("HALT\n");
179     goto finish;
180 }
181
182 ifex.valid = 0;
183 return;
184
185 finish:
186     printf("\n=== Simulation Results ===\n");
187     printf("Instructions: %d\n", instructions);
188     printf("Total branches: %d\n", total_branches);
189     printf("Correct predictions: %d\n", correct_predictions);
190     printf("Mispredictions: %d\n", mispredictions);
191     printf("Branch penalty: %d cycles\n",
192         branch_penalty_cycles);
193     printf("Total cycles: %d\n", cycles);
194     printf("CPI: %.2f\n",
195         (float)cycles / instructions);
196     exit(0);
197 }
198
199 /* ===== MAIN ===== */
200 int main() {
201     reset_cpu();
202     load_program("program.hex");
203
204     while (1) {
205         cycles++;
206         flush_pipeline = 0;
207
208         execute_stage();
209
210         if (flush_pipeline) {
211             ifex.valid = 0;
212             continue;
213         }
214
215         fetch_stage();
216     }
217     return 0;
218 }

```


12. Experiments to Perform

Students must run the same program with:

- No prediction
- Predict-not-taken
- 1-bit predictor

And compare:

- CPI
- Number of mispredictions
- Total penalty cycles

Sample Programs for Evaluation

To understand the effect of branch prediction, you must evaluate your simulator using the following two programs. Run each program under:

- No prediction
- Predict-Not-Taken
- 1-bit Branch Predictor

Compare CPI, mispredictions, and penalty cycles.

Sample Program 1: Loop Branch (Highly Predictable)

Purpose: This program demonstrates the benefit of branch prediction in loops, where the branch outcome is repetitive.

Assembly Code

```
; Initialize loop counter
MOVI 5          ; ACC = 5
STORE R0        ; R0 = 5

LOOP:
LOAD R0          ; ACC = R0
SUB R1           ; ACC = ACC - 1
STORE R0         ; R0 = R0 - 1
CMP R1           ; compare ACC with 0
JZ LOOP          ; branch taken until counter reaches 0
HALT
```

Branch Behavior

- The branch is taken repeatedly
- The branch is not taken only once to exit the loop

Expected Observations

- **No Prediction:** Pipeline flush on every iteration
- **Predict-Not-Taken:** Misprediction on every taken iteration
- **1-bit Predictor:** Only first and last iterations mispredicted

This program should show a **large CPI improvement** when using a 1-bit branch predictor.

Sample Program 2: If–Else Branch (Unpredictable)

Purpose: This program illustrates the limitations of simple branch predictors when branch outcomes change frequently.

Assembly Code

```
MOVI 0
STORE R0
```

```
LOAD R0
CMP R1
JZ THEN
ADD R2
JMP NEXT
```

```
THEN:
SUB R2
```

```
NEXT:
MOVI 1
ADD R0
STORE R0
```

```
CMP R0
JZ LOAD
HALT
```

Branch Behavior

- Branch outcome changes frequently
- No strong pattern is present

Expected Observations

- **Predict-Not-Taken:** Approximately 50% mispredictions
- **1-bit Predictor:** Frequent predictor flipping (thrashing)
- CPI improvement is limited compared to Sample Program 1

This program demonstrates that branch prediction works best when branch behavior is repetitive.

Comparison Table to Include in Report

Students must include a table similar to the following:

Program	Predictor	CPI	Mispredictions
Loop	No Prediction		
Loop	Predict-NT		
Loop	1-bit		
If-Else	Predict-NT		
If-Else	1-bit		

13. Learning Outcomes

After completing this assignment, you will understand:

- Why control hazards reduce performance
- How branch prediction improves CPI
- How real RISC processors speculate safely